BitPub Documento di Progettazione

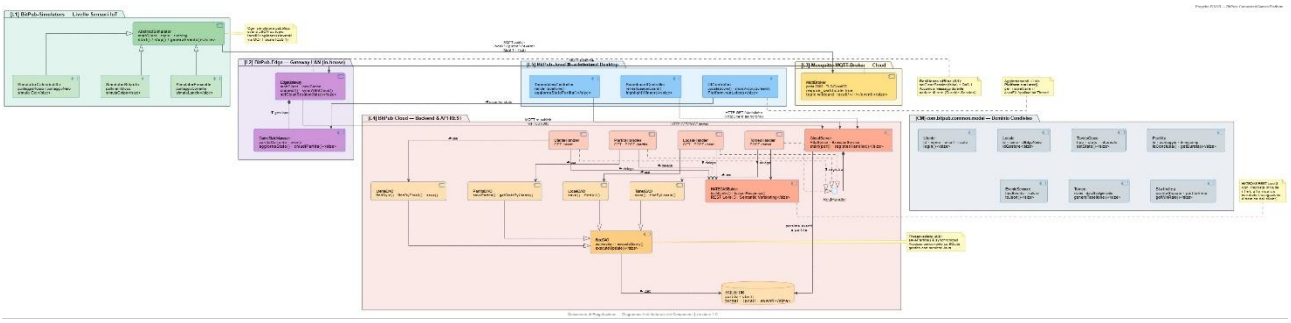
1. Diagramma architetturale

Scelta del Pattern Architetturale: Model-View-Controller (MVC) Distribuito Per la progettazione di BitPub, considerando i rigidi vincoli accademici (assenza di framework web di alto livello) e la natura IoT del sistema, il nostro gruppo ha scelto di adottare il pattern architetturale MVC (Model-View-Controller) riadattato per un'architettura distribuita (Edge-Cloud). Il nostro intento principale è definire fin da subito una netta separazione tra la logica di gestione dei dati, l'acquisizione hardware e la loro rappresentazione. In questa fase di design, riteniamo fondamentale disaccoppiare questi aspetti per garantire che lo sviluppo del backend Cloud (esposto via HttpServer) possa procedere indipendentemente dall'interfaccia grafica locale (JavaFX) e dai simulatori hardware. Questa scelta strategica mira ad assicurare la massima modularità del sistema e garantire la tolleranza ai guasti di rete (offline resilience).

Sulla base di questa analisi, abbiamo progettato l'architettura del sistema come segue:

- **View (Livello di Presentazione):** Sarà il componente dedicato all'interfaccia grafica nei locali fisici. Intendiamo implementare questo livello utilizzando file FXML (JavaFX), che definiranno la struttura visiva del tabellone segnapunti senza incorporare logica di business. La comunicazione per l'aggiornamento real-time avverrà strettamente tramite Platform.runLater() per garantire la fluidità del thread grafico.
- **Controller (Livello di Controllo):** È progettato per fare da intermediario. Nell'**Edge Node**, i controller JavaFX intercetteranno i payload MQTT in arrivo (tramite la libreria Eclipse Paho) e aggiorneranno la View. Nel **Server Cloud**, i controller saranno gestori puri (HttpHandler) che processeranno le richieste REST HTTP in modo state-less, validando i dati in ingresso e garantendo i principi HATEOAS e il Semantic Versioning.
- **Service Layer (Livello di Servizio):** Per mantenere puliti i Controller, è stato previsto l'inserimento di un Service Facade. L'intento è che il Controller deleghi a questo servizio l'esecuzione delle operazioni complesse, come il calcolo delle statistiche globali, il matchmaking dei tornei o la validazione dei JSON generati dai sensori fisici (Calciobalilla, Freccette, Biliardo).

- **Model (Livello Dati):** Rappresenterà lo stato del sistema e la logica di accesso ai dati. La nostra progettazione prevede due componenti distinti:
    - **Domain Model:** Classi POJO (come Partita, Utente, Tavolo) serializzate e deserializzate tramite Google GSON, che modelleranno le entità del dominio.
    - **DAO (Data Access Objects):** Il layer nel Server Cloud che si occuperà della persistenza, eseguendo le query SQL e mappando i risultati dal database SQLite agli oggetti del dominio. L'accesso al DB sarà strettamente protetto da blocchi synchronized per gestire la concorrenza garantendo la thread-safety. Nel nodo Edge, il layer Model si appoggerà alla persistenza temporanea garantita dalla *Durable Session* MQTT (setCleanSession(false)).

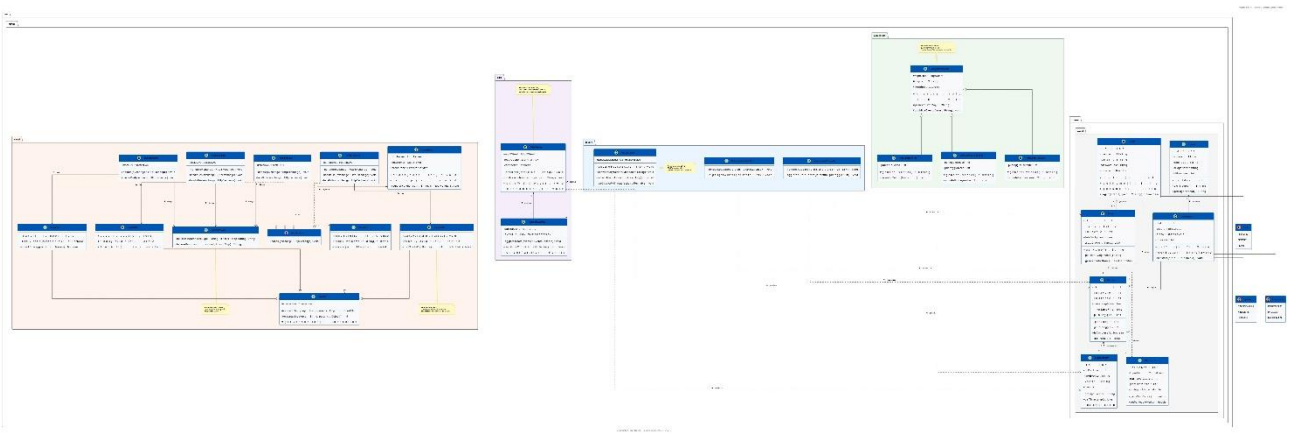


2. Diagramma delle classi di dominio

Modello di dominio

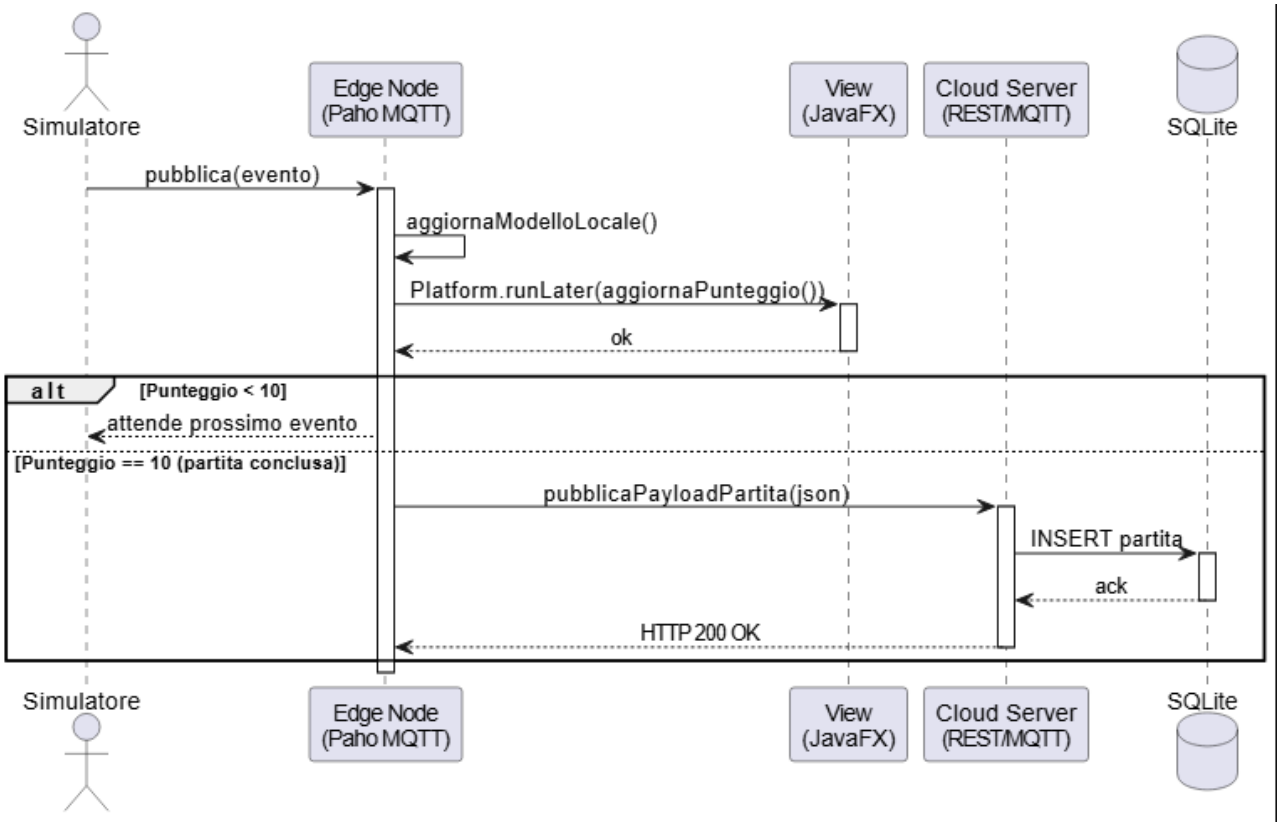
Classe	Attributi Principali	Responsabilità
Locale	id, nome, indirizzo, id_edge_node	Rappresenta la struttura fisica autorizzata (sala giochi, bar). Raggruppa i giochi presenti.
Utente	id, nome, email, password, ruolo (Giocatore/Gestore/Admin)	Gestisce l'identità e i permessi di accesso alle API della piattaforma.
TavoloGioco	id, id_locale, tipo (Calciobalilla, Freccette, Biliardo), stato_sensore	Rappresenta l'entità fisica che genera gli eventi IoT tramite MQTT.

Classe	Attributi Principali	Responsabilità
Partita	id, id_tavolo, timestamp_inizio, timestamp_fine, punteggio_finale, stato	Registra la singola sessione di gioco, raccogliendo l'esito finale dagli Edge Node.
EventoSensore	id, id_partita, tipo_evento (es. "Gol", "Buca", "Settore 20"), timestamp	Rappresenta il singolo evento atomico IoT scaturito dall'hardware simulato.
Torneo	id, nome, data_svolgimento, id_tavolo, id_gestore, stato	Modella una competizione creata dai gestori, gestendo le iscrizioni e le fasi a eliminazione.
Statistica	id_utente, id_tipo_gioco, partite_giocate, win_rate, punteggio_medio	DTO (Data Transfer Object) aggregato calcolato dal Cloud per servire i client REST.

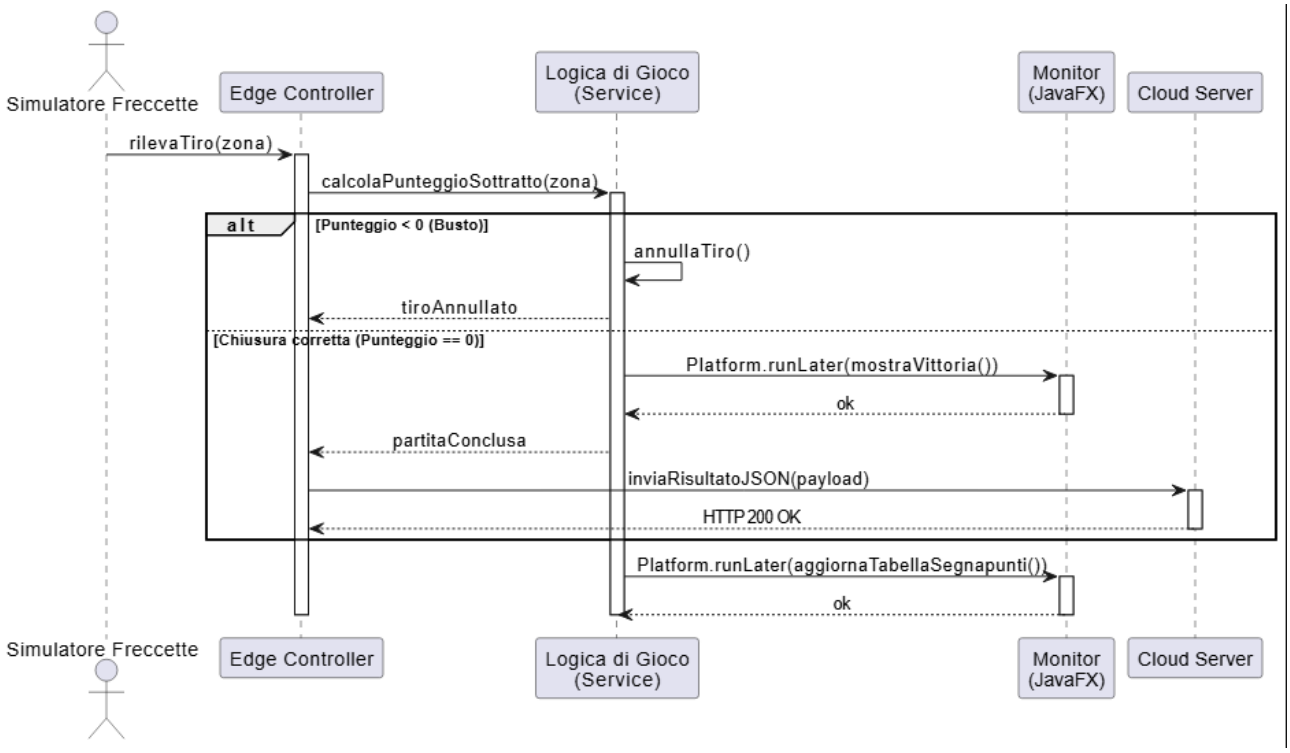


3. Diagramma di sequenza

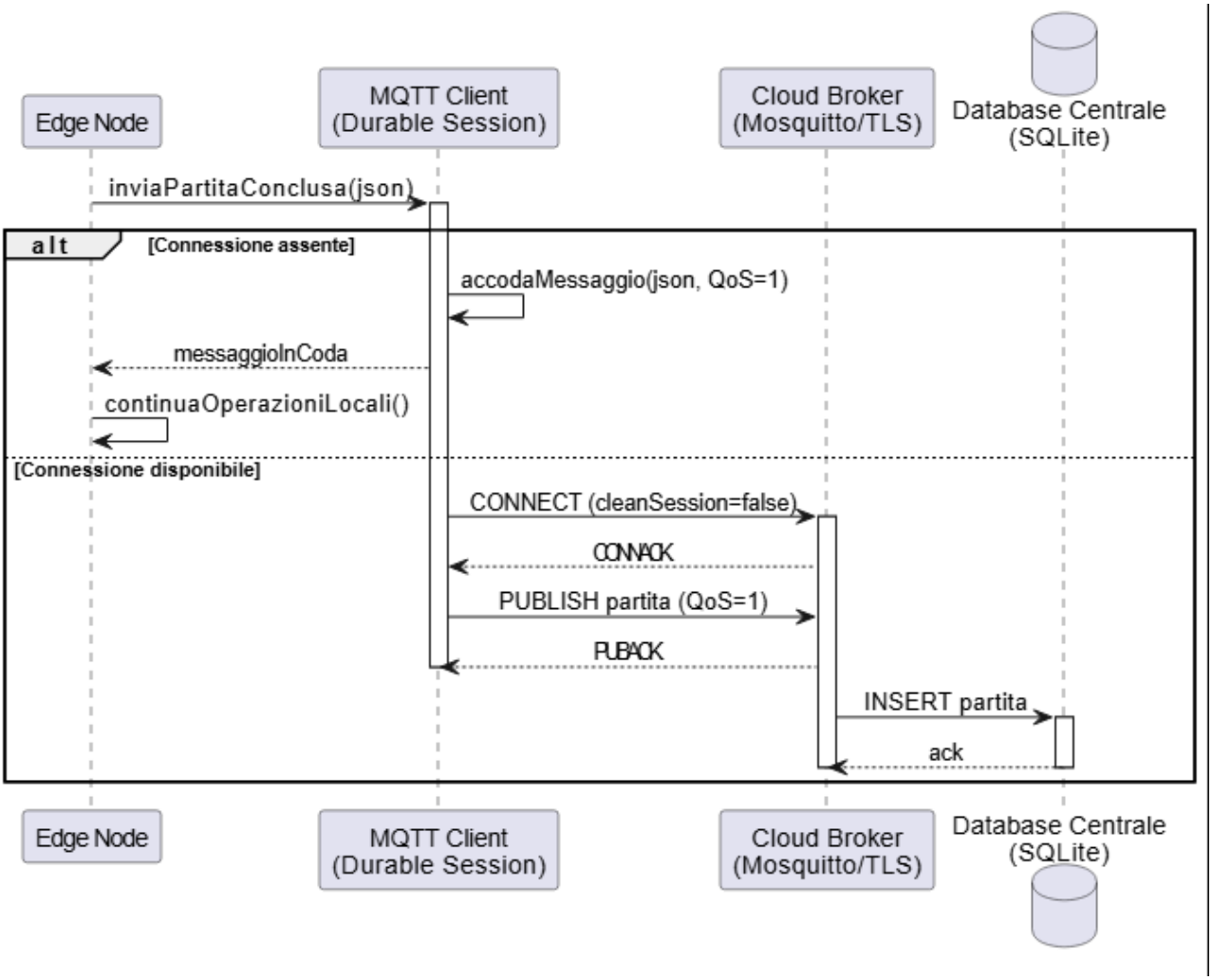
Caso d'uso 1 Elaborazione Eventi Calciobalilla



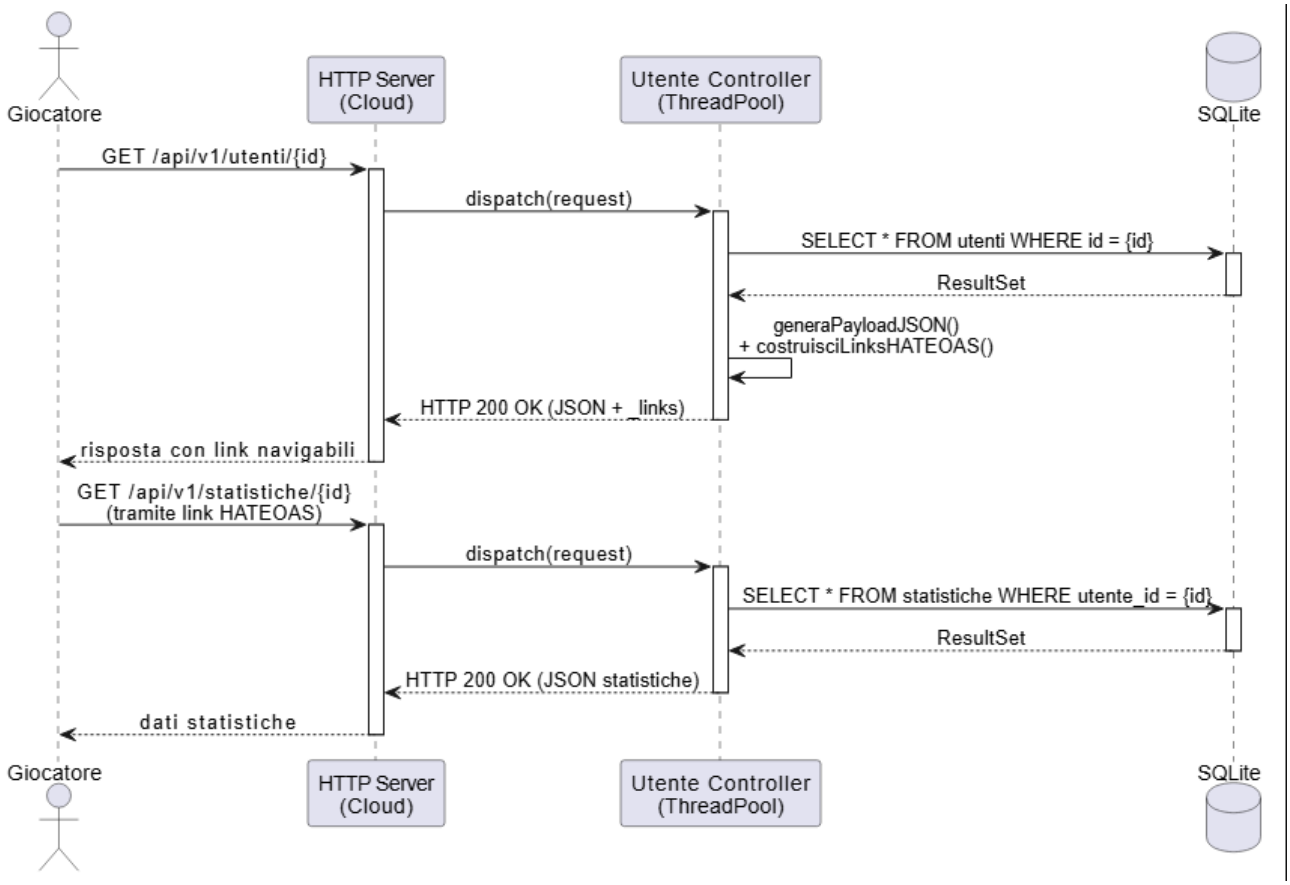
Caso D'uso 2 Gestione Partita Freccette e UI Locale



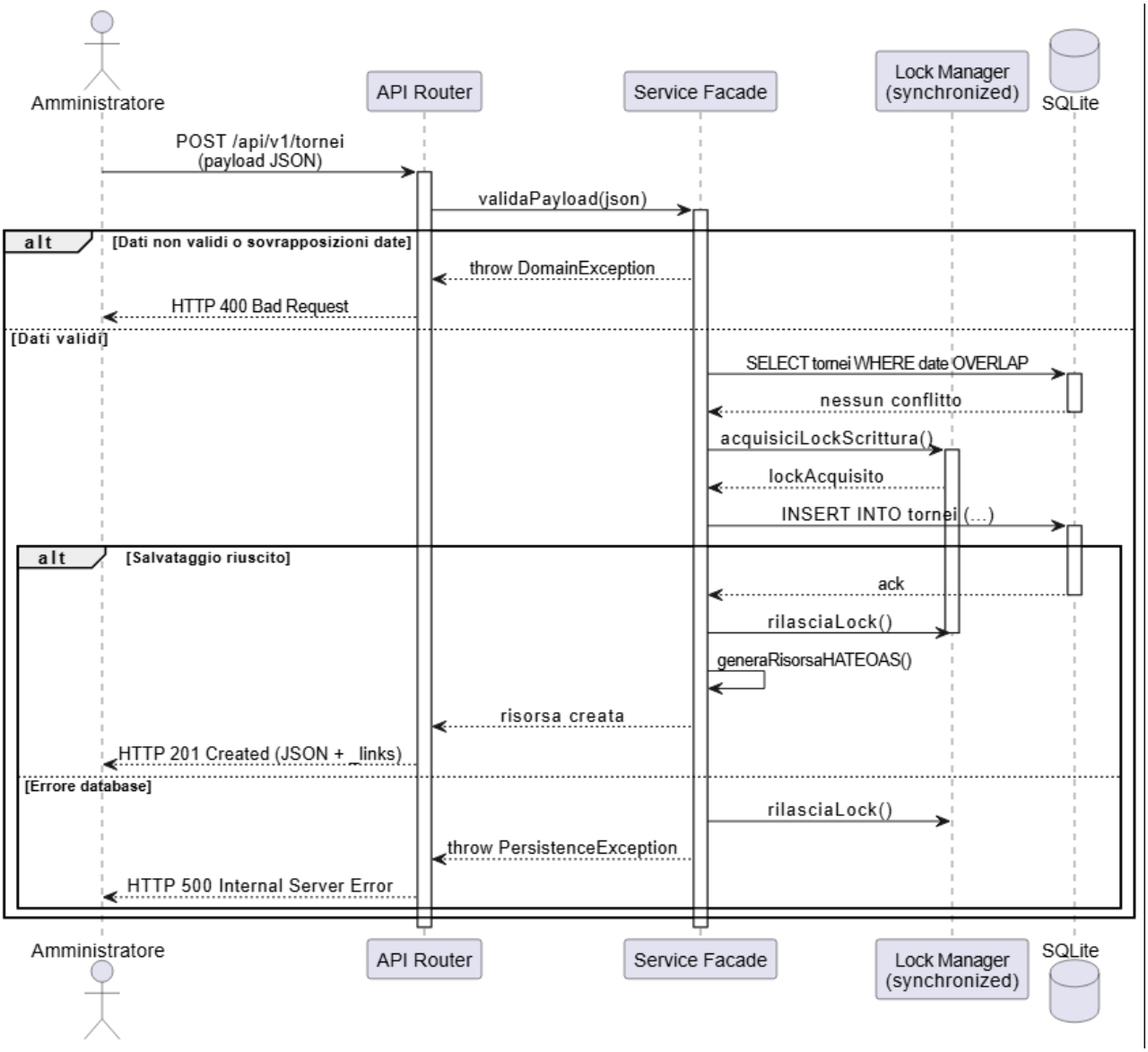
Caso D’uso 3 Sincronizzazione Dati Biliardo Offline/Online



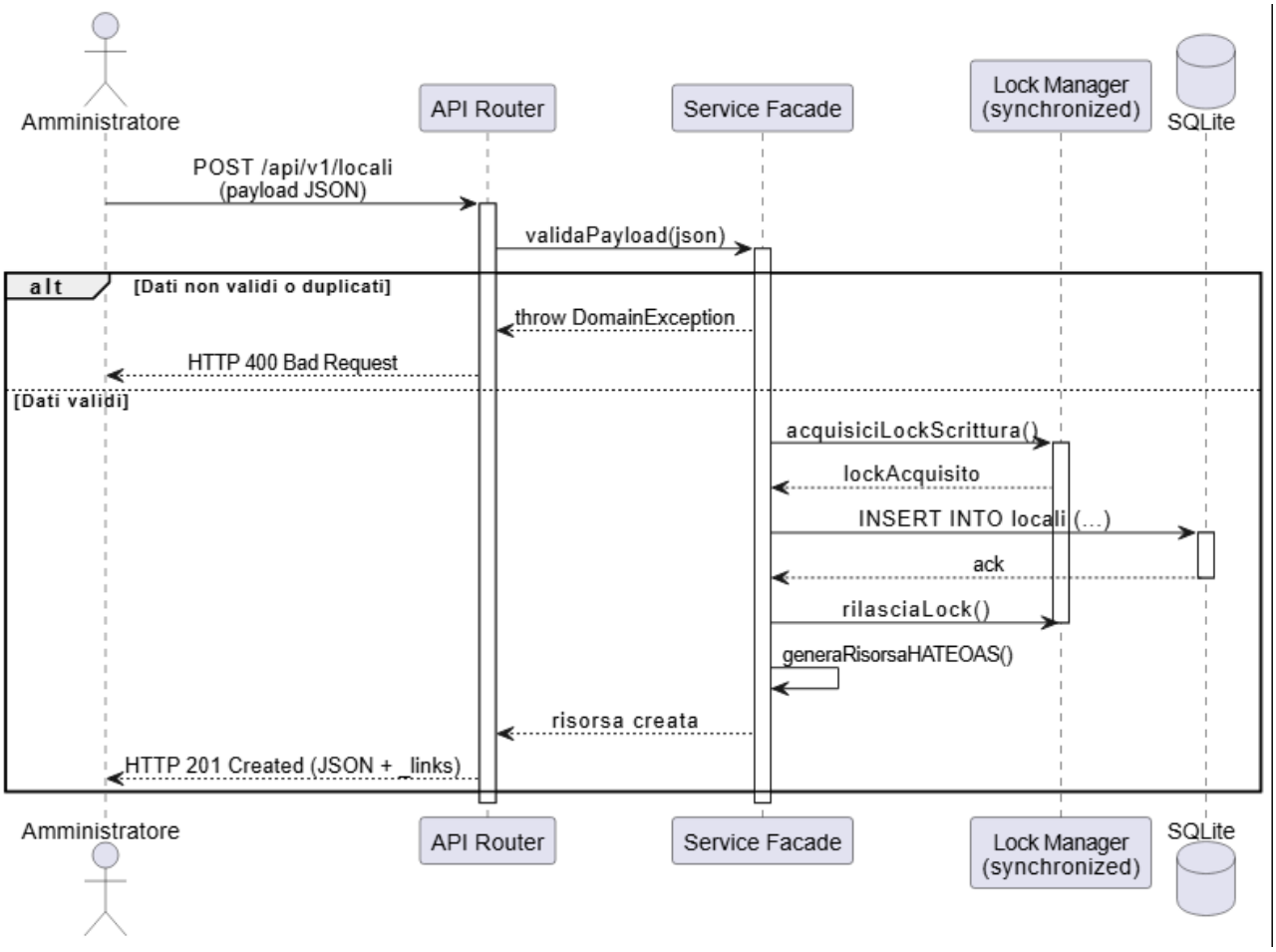
Caso D'uso 4: Consultazione statistiche HATEOAS



Caso D'uso 5: Creazione e Gestione Torneo(Singolo/Squadre)

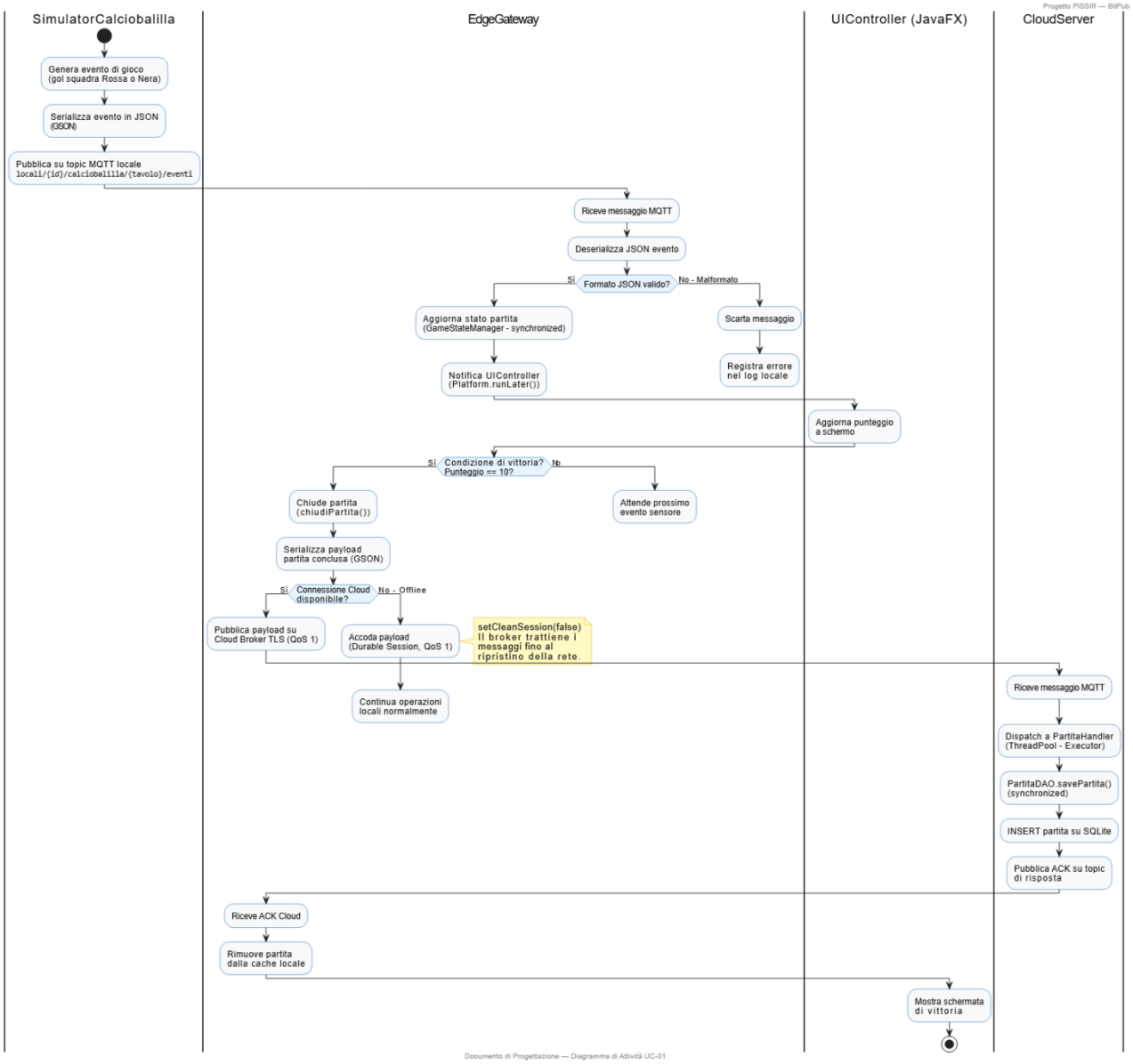


Caso D'uso 6: Configurazione Anagrafica

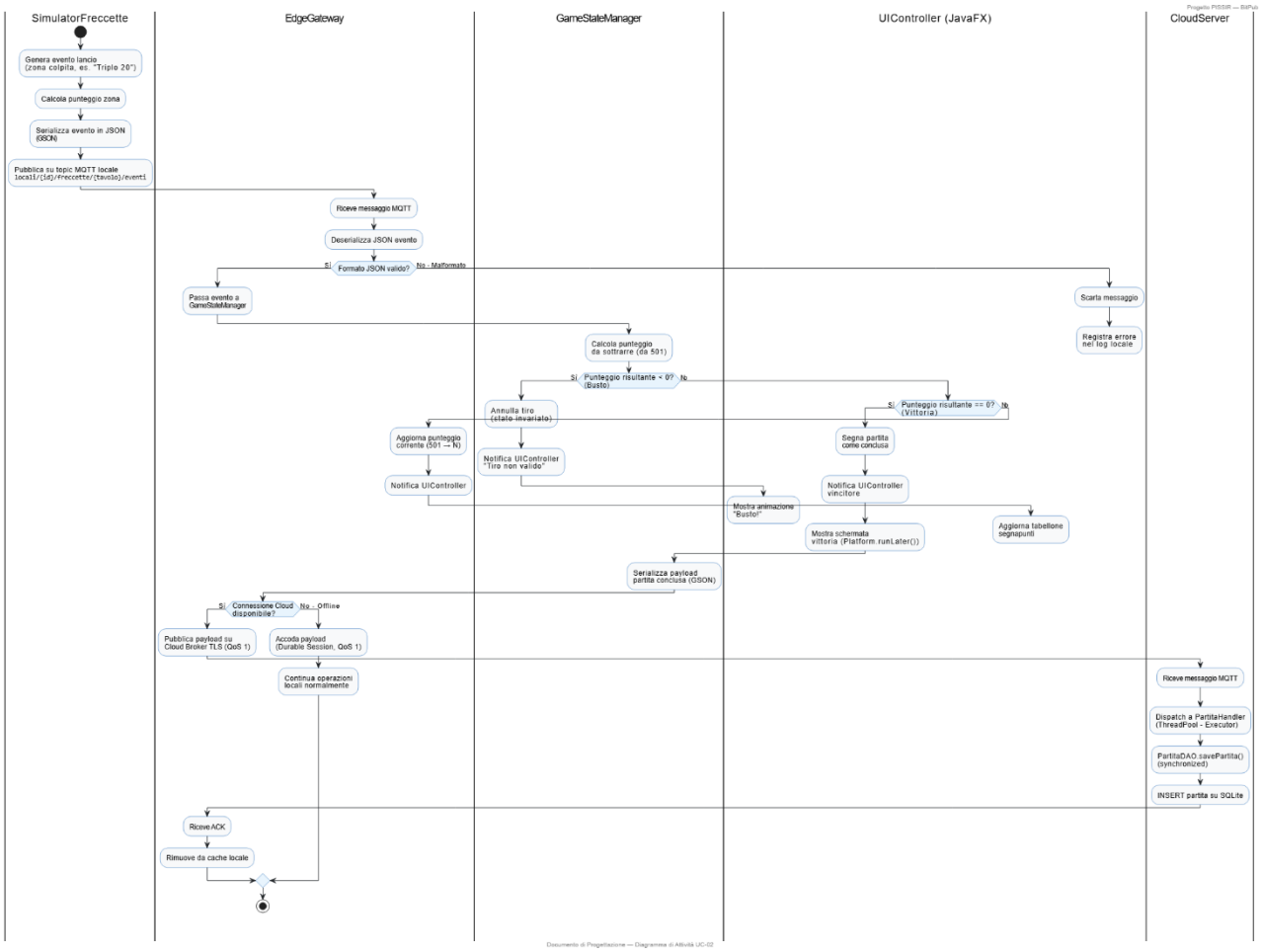


4. Diagramma di Attività

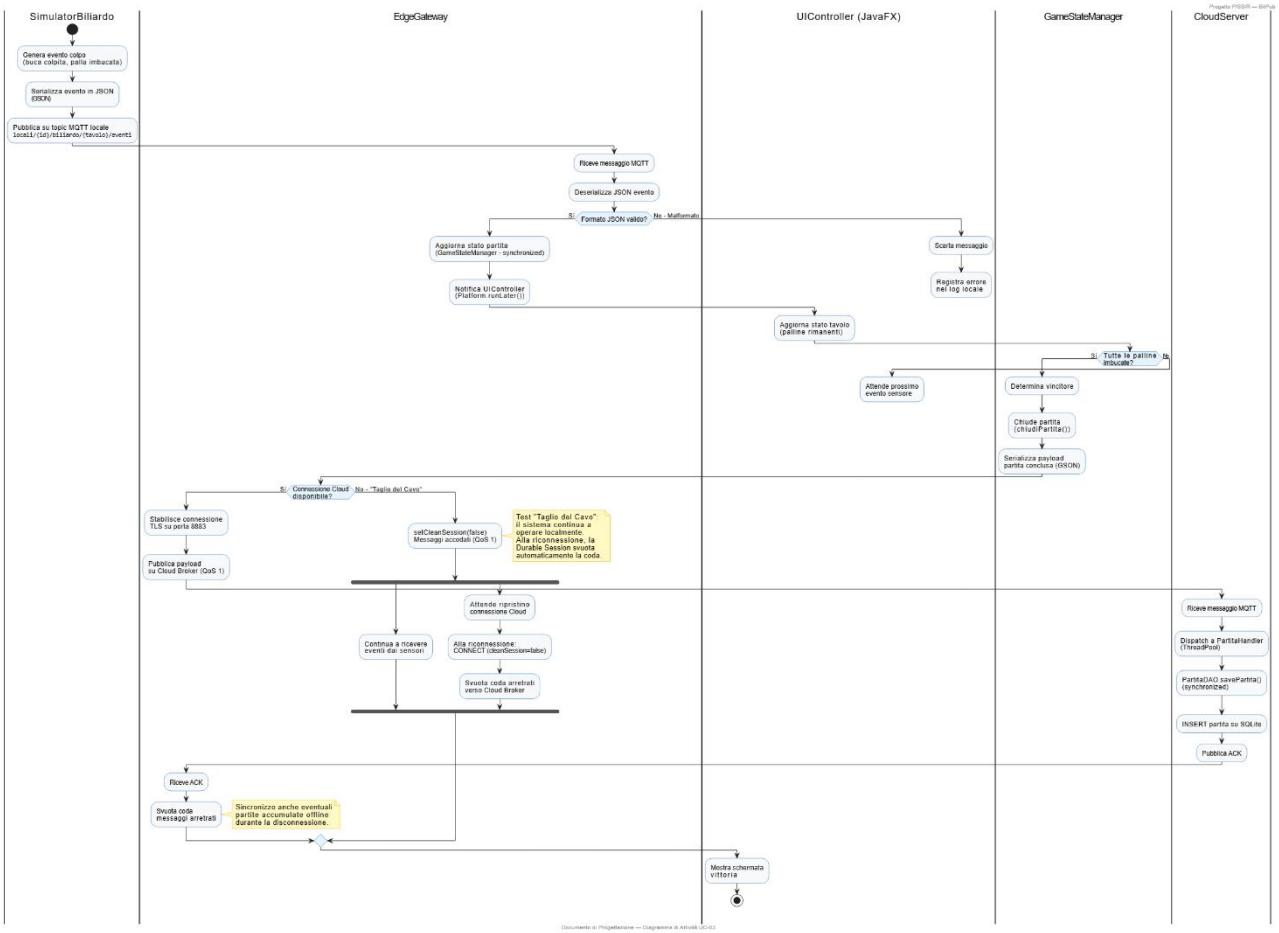
Caso D'uso 1 Elaborazione e invio punteggi partite calciobalilla



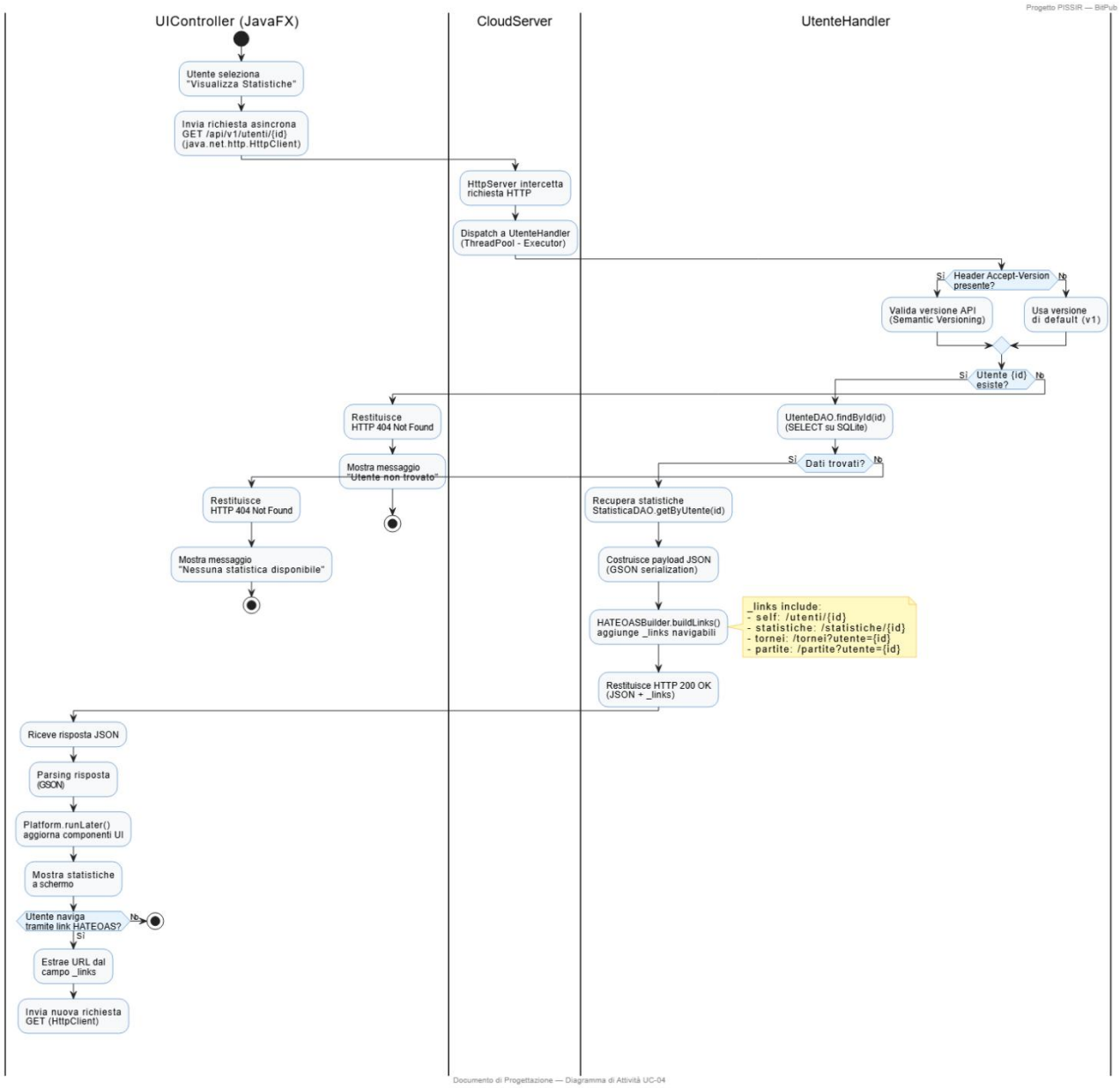
Caso D’uso 2 Gestione Partita Freccette e UI Locale



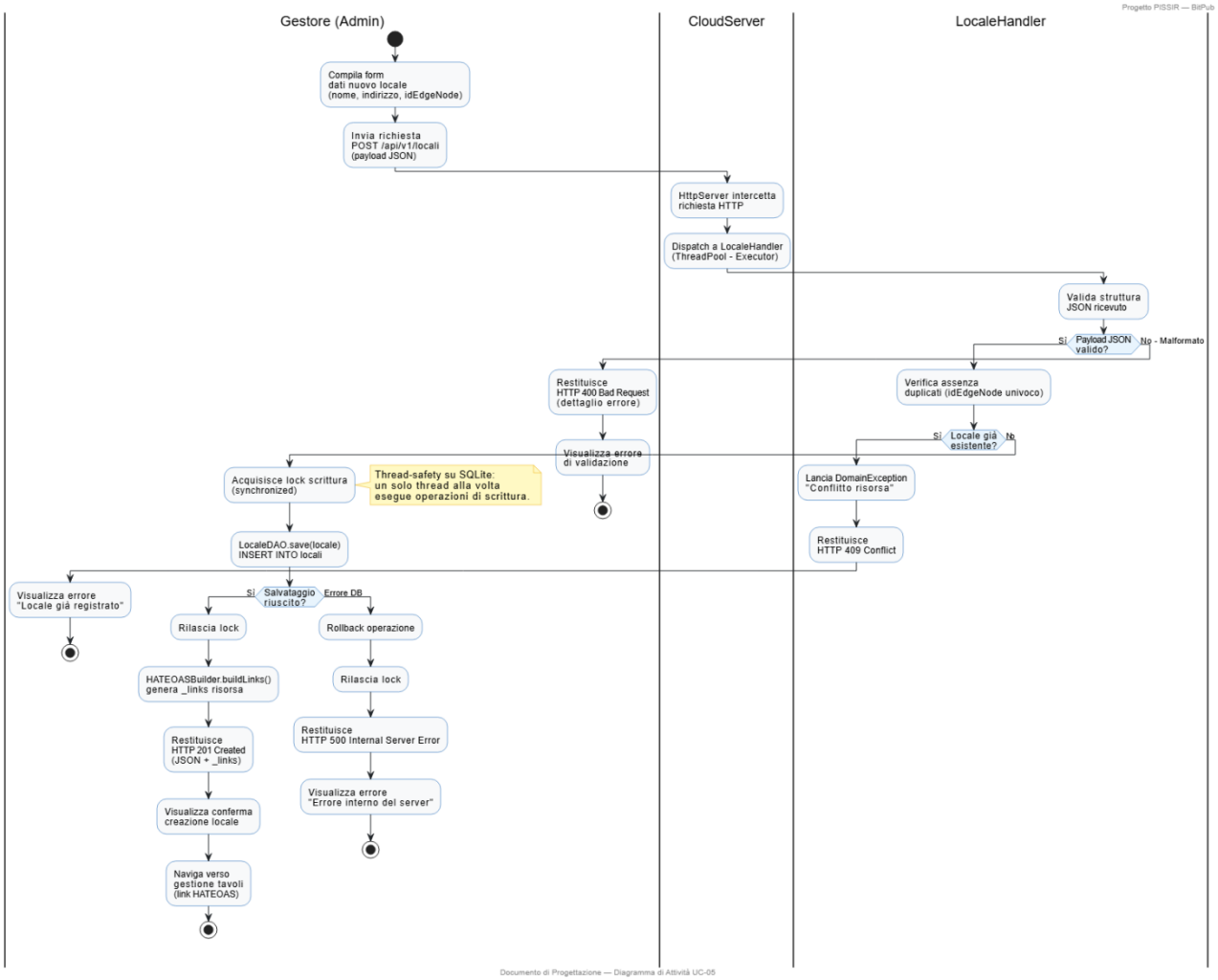
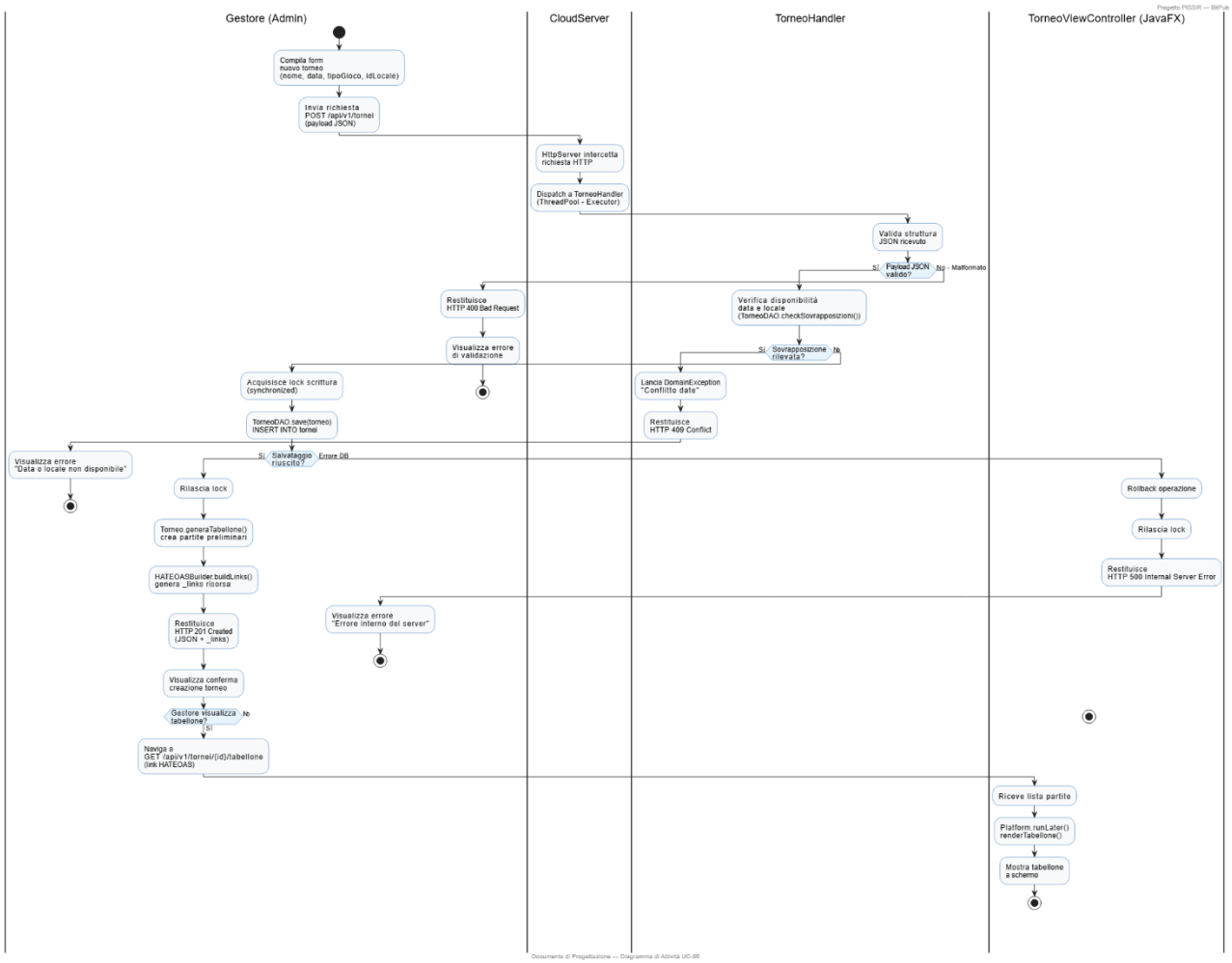
Caso D’uso 3 Sincronizzazione Dati Biliardo Offline/Online



Caso D’uso 4: Consultazione statistiche HATEOAS



Caso D’uso 5: Creazione e Gestione Torneo(Singolo/Squadre)



Caso D’uso 6: Configurazione Anagrafica